Modules and User-Defined Modules

---

1. Introduction to Modules

What is a Module?

A **module** is a file containing Python code, including functions, classes, and variables, saved with a `.py` extension.

Modules help in:

- Organizing code into logical sections
- Reusing code across multiple programs
- Making code more readable and maintainable
- Supporting collaborative development

---

2. Types of Modules in Python

1. **User-Defined Modules** – Created by the programmer.
2. **Built-in Modules** – Provided by Python (e.g., `math`, `os`) – *(covered in a separate guide)*.
3. **External Modules** – Third-party modules installed using pip.

This guide focuses on **user-defined modules and functions**.

---

3. Importing Modules in Python

Python provides several ways to import modules:

a. Import Entire Module

```python
import my_module
print(my_module.function_name())
```

b. Import Specific Items from a Module

```python
from my_module import function_name
print(function_name())
```

c. Import with an Alias

```python
import my_module as mm
print(mm.function_name())
```

d. Avoid (Not Recommended): Import All Items

```python
from my_module import *
# May cause name conflicts
```

---

4. User-Defined Modules

A **user-defined module** is any Python file created by the user to store reusable code.

Step-by-Step Example

Step 1: Create the Module

Create a file named `my_module.py`:

```python
# my_module.py
```

```python
def greet(name):
    return f"Hello, {name}!"

pi = 3.14159
```

Step 2: Use the Module in Another File

Create a file named `main.py`:

```python
# main.py

import my_module

# Use function from module
print(my_module.greet("Alice"))

# Use variable from module
print("Value of Pi:", my_module.pi)
```

Importing Specific Functions

```python
from my_module import greet

print(greet("Bob"))  # No need to prefix with module name
```

Using an Alias

```python
import my_module as m

print(m.greet("Charlie"))
```

5. Python Module Search Path

When importing a module, Python searches in the following locations:

1. Current directory
2. Directories listed in the `sys.path` variable

To view or modify the search path:

```python
import sys
print(sys.path)

# Add custom path
sys.path.append("C:/path/to/your/modules")
```

6. Using `if __name__ == "__main__"` in Modules

This conditional prevents specific code in a module from running when the module is imported.

```python
# my_module.py

def greet(name):
    return f"Hello, {name}!"

if __name__ == "__main__":
    print(greet("Alice"))  # Executes only when this file is run directly
```

- If you run: `python my_module.py` → the block executes
- If you import it in another file → the block is ignored

7. Installing and Using External Modules

To use a third-party module, install it using pip:

```
Python
pip install requests
```

Example usage:

```
Python
import requests
response = requests.get("https://www.example.com")
print(response.status_code)
```